

K. J. Somaiya College of Engineering, Mumbai-77

(Autonomous College Affiliated to University of Mumbai)

Batch: B1

Roll No.: 16010122158

Experiment / Assignment / Tutorial No. 2

TITLE: Project Metric estimations for Mini Project

AIM: To enable the students learn different techniques for performing software size and cost estimation

Expected Course outcome of Experiment:

Books/ Journals/ Websites referred:

1. Roger Pressman, "Software Engineering", sixth edition, Tata McGraw Hill.
2. http://sunset.usc.edu/csse/research/COCOMOII/cocomo_main.html
3. http://sunset.usc.edu/research/COCOMOII/expert_cocomo/expert_cocomo2000.html

Pre Lab/ Prior Concepts:

Software projects have tendency of going past their deadline, going over budget, or both. The problem lies in the estimation of the amount of effort required for the development of a project. The cost estimation is usually dependent upon the size estimate of the project, which may use lines of code or function points as metrics. There are several different techniques for performing software cost estimation, including expert judgement and algorithmic models. Estimation by expert judgement is a common way of estimating the effort required for a project. Unfortunately, this method of estimation does not emphasize re-estimation during the project life cycle, which is an important part of project tracking, because it allows the estimates to be improved during the project life cycle. The quality of a cost estimation model is not so much attributed to the initial estimate, but rather the speed at which the estimates converges to the actual cost of the project. COCOMO is a popular algorithmic model for cost estimation whose cost factors can be tailored to the individual development environment, which is important for the accuracy of the cost estimates. More than one method of cost estimation should be done so that there is some comparison available for the estimates. This is especially important for unique projects. Cost estimation must

K. J. Somaiya College of Engineering, Mumbai-77

(Autonomous College Affiliated to University of Mumbai)

be done more diligently throughout the project life cycle so that in the future there are fewer surprises and unforeseen delays in the release of a product.

Estimation of size and cost of the developing project is required for the following major decision situations

- Financial decisions involving a software development effort
- Setting project budgets and schedules as a basis for planning and control
- Deciding on or negotiating tradeoffs among software cost, schedule, functionality, performance or quality factors
- Making software cost and schedule risk management decisions
- Deciding which parts of a software system to develop, reuse, lease, or purchase
- Making legacy software inventory decisions: what parts to modify, phase out, outsource, etc
- Deciding how to implement a process improvement strategy, such as that provided in the SEI CMM

Defining Cost estimation:

Cost estimation can be defined as the approximate judgement of the costs for a project. Cost estimation will never be an exact science because there are too many variables involved in the calculation for a cost estimate, such as human, technical, environmental, and political. Furthermore, any process that involves a significant human factor can never be exact because humans are far too complex to be entirely predictable. Furthermore, software development for any fair-sized project will inevitably include a number of tasks that have complexities that are difficult to judge because of the complexity of software systems.

Cost estimation is usually measured in terms of effort. The most common metric used is person months or years (or man months or years). The effort is the amount of time for one person to work for a certain period of time. It is important that the specific characteristics of the development environment are taken into account when comparing the effort of two or more projects because no two development environments are the same. A clear example of differences in development environments are the amount of time people work in different countries; the typical workweek in North America is 40 hours per week, while in Europe the typical workweek is 35 hours per week. Thus, when comparing a project from North America with a project from Europe, a conversion factor would have to be used to allow for an accurate comparison. Different variables can be used for cost estimation, which leads to a difficulty when comparing projects if standard models or tools are not used. For example, a cost estimate can include factors from management, development (e.g., training, quality assurance), and other areas specific to an organization.

K. J. Somaiya College of Engineering, Mumbai-77

(Autonomous College Affiliated to University of Mumbai)

Estimator:

The people who do the cost estimates could be either directly or indirectly responsible for the implementation for a project, such as a developer or manager, respectively. Someone who has knowledge of the organization and previous projects could use an analogy-based approach to compare the current project with previous projects, which is a common method of estimation for small organizations and small projects. The historical data is often limited to the memory of the estimator. In this case, the estimator would need to be experienced and would likely have been with the company for awhile.

Some people believe it is better if the estimates are done by outsiders so that there is less chance of bias. It is true that people outside an organization will likely have to deal with fewer company politics than people within the organization. For example, the developer for a company may want to please the manager and so give an estimate that is overly-optimistic. The disadvantage of having an outside estimate is that the person would have less knowledge of the development environment, especially if the person is from outside the company. An empirical method of estimation would then be required, such as the Constructive Cost Model (COCOMO). Empirical methods of estimation can be used by all types of estimators. There may be some resistance to using an empirical method of estimation because there may be some question on whether a model could outperform an expert. People who are accurate estimators are rare in our experience, and so it is best to get the opinion of several people or tools.

Cost estimation using different COCOMO models:

K. J. Somaiya College of Engineering, Mumbai-77

(Autonomous College Affiliated to University of Mumbai)



FP CALCULATOR

Domain Characteristic Table

MEASUREMENT PARAMETER	COUNT (value >= 0)	WEIGHTING FACTOR		
		Simple	Average	Complex
Number of User Input	15	☒	☐	☐
Number of User Outputs	10	☒	☐	☐
Number of User Inquiries	5	☒	☐	☐
Number of Files	10	☒	☐	☐
Number of External Interfaces	6	☒	☐	☐

[Complexity Adjustment Table](#) | [FP Calculation](#)

Complexity Adjustment Table

ITEM	COMPLEXITY ADJUSTMENT QUESTIONS	SCALE					
		No Influence				Essential	
		0	1	2	3	4	5
1	Does the system require reliable backup and recovery?	☐	☐	☐	☐	☐	☒
2	Are data communications required?	☐	☐	☐	☐	☒	☐
3	Are there distributed processing functions?	☐	☐	☐	☒	☐	☐
4	Is performance critical?	☐	☐	☐	☐	☐	☒

K. J. Somaiya College of Engineering, Mumbai-77

(Autonomous College Affiliated to University of Mumbai)

5	Will the system run in an existing, heavily utilized operational environment?	☐ ☐ ☐ ☐ ☒ ☐
6	Does the system require on-line data entry?	☐ ☐ ☐ ☐ ☒ ☐
7	Does the on-line data entry require the input transaction to be built over multiple screens or operations?	☐ ☐ ☐ ☒ ☐ ☐
8	Are the master files updated on-line?	☐ ☐ ☐ ☐ ☒ ☐
9	Are the inputs, outputs, files or inquiries complex?	☐ ☐ ☐ ☒ ☐ ☐
10	Is the internal processing complex?	☐ ☐ ☐ ☒ ☐ ☐
11	Is the code to be designed reusable?	☐ ☐ ☐ ☐ ☒ ☐
12	Are conversion and installation included in the design?	☐ ☒ ☐ ☐ ☐ ☐
13	Is the system designed for multiple installations in different organizations?	☐ ☐ ☐ ☒ ☐ ☐
14	Is the application designed to facilitate change and ease of use by the user?	☐ ☐ ☐ ☐ ☒ ☐

[Domain Characteristic Table](#) | [FP Calculation](#)

FP Calculation

NOTE: For any updates made on any of the entries, always click the 'Calculate Function Points' button to recalculate function points value.

[Reset / Clear all form entries](#) [Calculate Function Points](#)

RESULT	
PROJECT FUNCTION POINTS	229.99999999999997

[Top of Page](#) | [Domain Characteristic Table](#) | [Complexity Adjustment Table](#)

MANUAL CALCULATION OF FP

Measurement Parameter	Optimistic	Likely	Pessimistic	Estimate Count	Weight	FP Count
Number of User Inputs	12	15	18	15	3	45
Number of User Outputs	8	10	12	10	4	40
Number of	3	5	7	5	3	15

K. J. Somaiya College of Engineering, Mumbai-77

(Autonomous College Affiliated to University of Mumbai)

User Inquiries						
Number of Files	8	10	12	10	7	70
Number of External Interfaces	4	6	8	6	5	30
Count	200					

Factor	Value
Backup and Recover	5
Data Communications	4
Distributed Processing	3
Performance Critical	5
Existing Operating Environment	4
Online Data Entry	4
Input Transactions over Multiple Screens	3
Master File Updated online	4
Information Domain Values Complex	3
Internal Processing Complex	3
Code Designed for Reuse	4
Conversion/Installation in Design	2
Multiple Installations	3
Applications Designed for Change	4

Estimated Number of FP is

$$\text{Count Total} * (0.65 + 0.01 * \sum Fi) = 200 * (0.65 + 0.01 * 51) = 232$$

Cost Per FP is \$1000

Average Productivity is 6.5

K. J. Somaiya College of Engineering, Mumbai-77

(Autonomous College Affiliated to University of Mumbai)

Thus the FP estimate for this project is:

$$232/6.5 = 36 \text{ person/ month}$$

$$372 * 1000 = \$372000$$

COCOMO MODEL:-

Safari File Edit View History Bookmarks Window Help

Not Secure — softwarecost.org

Failed to open page Welcome to XAMPP BEVERLY AUTOMOTIVES COCOMO II - Constructive Cost Model

COCOMO II - Constructive Cost Model

Monte Carlo Risk Auto Calculate

Software Size Sizing Method

SLOC

	% Design Modified	% Code Modified	% Integration Required	Assessment and Assimilation (0% - 8%)	Software Understanding (0% - 50%)	Unfamiliarity (0-1)
New	19346					
Reused	0	0	15	0		
Modified	0	0	0	0	30	0.2

Software Scale Drivers

Precedentedness	Nominal	Architecture / Risk Resolution	Nominal	Process Maturity	Nominal
Development Flexibility	Nominal	Team Cohesion	Nominal		

Software Cost Drivers

Product	Personnel	Platform			
Required Software Reliability	Nominal	Analyst Capability	Nominal	Time Constraint	Nominal
Data Base Size	Nominal	Programmer Capability	Nominal	Storage Constraint	Nominal
Product Complexity	Nominal	Personnel Continuity	Nominal	Platform Volatility	Nominal
Developed for Reusability	Nominal	Application Experience	Nominal	Project	
Documentation Match to Lifecycle Needs	Nominal	Platform Experience	Nominal	Use of Software Tools	Nominal
		Language and Toolset Experience	Nominal	Multisite Development	Nominal
				Required Development Schedule	Nominal

Maintenance

Software Labor Rates

Cost per Person-Month (Dollars) 3500

Results

Documentation Match to Lifecycle Needs Platform Experience Use of Software Tools

Language and Toolset Experience Multisite Development

Required Development Schedule

Maintenance

Software Labor Rates

Cost per Person-Month (Dollars) 3500

Results

Software Development (Elaboration and Construction) Staffing Profile

Effort = 76.4 Person-months
Schedule = 14.6 Months
Cost = \$267472

Total Equivalent Size = 19346 SLOC
Effort Adjustment Factor (EAF) = 1.00

Acquisition Phase Distribution

Phase	Effort (Person-months)	Schedule (Months)	Average Staff	Cost (Dollars)
Inception	4.6	1.8	2.5	\$16046
Elaboration	18.3	5.5	3.4	\$64193
Construction	58.1	9.1	6.4	\$203279
Transition	9.2	1.8	5.0	\$32097

Software Effort Distribution for RUP/MBASE (Person-Months)

Phase/Activity	Inception	Elaboration	Construction	Transition
Management	0.6	2.2	5.8	1.3
Environment/CM	0.5	1.5	2.9	0.5
Requirements	1.7	3.3	4.6	0.4
Design	0.9	6.6	9.3	0.4
Implementation	0.4	2.4	19.7	1.7
Assessment	0.4	1.8	13.9	2.2
Deployment	0.1	0.6	1.7	2.8

can't open file

K. J. Somaiya College of Engineering, Mumbai-77
(Autonomous College Affiliated to University of Mumbai)

FUNCTION	OPTIMISTIC LOC	MOST LIKELY LOC	PESSIMISTIC LOC	ESTIMATED LOC
UICF	2000	2200	2400	2200
2DGA	3000	3200	3400	3200
3DGA	3500	3700	3900	3800
DBM	2500	2700	2900	2700
CGDF	2800	3000	3200	3000
PC	1500	1700	1900	1700
DAM	2500	2700	2900	2700
ESTIMATED LOC				19300

Historical data indicates that organisational average productivity for systems of this type is 250 LOC per month and cost per LOC is \$13.

The LOC estimate for this project is:-

$$19300/250 = 77.2$$

$$19300 * 13 = 250900$$

Conclusion:

In this experiment, the COCOMO model performed excellently for calculating project effort, duration, and staffing requirements. By accounting for factors including project size, complexity, and team experience, precise estimates were generated. A software project is said to be an organic type if the team size required is adequately small, the problem is well understood and has been solved in the past and also the team members have a nominal experience regarding the problem.

Post Lab Descriptive Questions

1. Explain COCOMO II model

COCOMO II, which stands for Constructive Cost Model II, is an enhanced version of the original COCOMO model. It's a software cost estimation model designed to provide more accurate and detailed estimates for software project planning and management.

K. J. Somaiya College of Engineering, Mumbai-77

(Autonomous College Affiliated to University of Mumbai)

COCOMO II takes into account various factors that impact the cost, effort, and schedule of software development projects, providing a systematic way to analyze and predict these aspects.

References to this model typically call it COCOMO II. In 1997 COCOMO II was developed and finally published in 2000. COCOMO II is the successor of COCOMO I and is better suited for estimating modern software development projects. It provides more support for modern software development processes and an updated project database.

COCOMO II is tuned to modern software life cycles. The original COCOMO model has been very successful, but it doesn't apply to newer software development practices as well as it does to traditional practices. COCOMO II targets modern software projects, and will continue to evolve over the next few years.

The following are the various COCOMO submodels:

Application Composition Model (COCOMO II-ACM): This submodel is used during the earliest stages of project planning when only limited information about the project is available. It estimates project size based on lines of code or function points and employs a set of scaling factors to derive a preliminary estimate of effort and schedule.

Early Design Model (COCOMO II-EDM): As the project advances and more details become available, the Early Design Model comes into play. It considers characteristics such as the software architecture, development process, and the team's experience. This submodel provides a more refined estimate by factoring in these details.

Post-Architecture Model (COCOMO II-Post-Arch): Once the software architecture is established, this submodel is used to provide even more detailed estimates. It takes into account factors like reuse of existing components, team experience, project constraints, and additional cost drivers that influence the project's complexity and resource requirements.

COCOMO II works by using mathematical formulas that combine the size of the project, the chosen submodel, and the values of the effort multipliers. The result is an estimation of effort (measured in person-months), schedule (in months), and cost (in terms of resources and finances) required to complete the project.

2. Compare the merits & limitations of basic COCOMO model & COCOMO II model.

Basic COCOMO Model:

Merits:

K. J. Somaiya College of Engineering, Mumbai-77

(Autonomous College Affiliated to University of Mumbai)

- **Simplicity:** The Basic COCOMO model is straightforward and easy to understand. It uses a simple mathematical formula based on lines of code (LOC).
- **Early Estimation:** Provides quick and early estimates of effort, time, and cost, which are useful in the initial stages of project planning.
- **Historical Basis:** Developed based on historical data from numerous projects, providing a solid empirical foundation.

Limitations:

- **Limited Scope:** The model is based on a limited set of project types and does not handle modern software development practices well.
- **Static:** Assumes that the project characteristics remain static, which is rarely the case in real-world projects.
- **Limited Accuracy:** Less accurate for complex, large, or high-risk projects due to its simplicity and reliance on LOC.
- **Single Input:** Relies primarily on lines of code as the primary input, which may not reflect all aspects of software complexity and effort.

COCOMO II Model:

Merits:

- **Flexibility:** Accounts for modern software development practices and various types of software projects.
- **Detailed:** Considers multiple cost drivers and factors such as software reuse, product complexity, and development environment.
- **Phased Estimation:** Provides estimates for different stages of the software lifecycle, from early design to post-architecture.
- **Calibration:** Can be calibrated to reflect the specific conditions and historical data of an organization, improving accuracy.

Limitations:

- **Complexity:** More complex to use and understand compared to the Basic COCOMO model, requiring more detailed inputs and data.
- **Data Requirement:** Requires extensive and accurate data for calibration, which may not always be available.
- **Effort Intensive:** More time-consuming to apply due to the need for detailed inputs and analysis.

K. J. Somaiya College of Engineering, Mumbai-77

(Autonomous College Affiliated to University of Mumbai)

- **Learning Curve:** Requires a higher level of expertise and understanding to use effectively.

3. Briefly explain the various types of for efforts & cost estimation techniques used in Software Engineering

1. Expert Judgment:

- **Description:** Relies on the expertise and experience of individuals or groups to estimate effort and cost.
- **Merits:** Quick, leverages deep domain knowledge, flexible.
- **Limitations:** Subjective, may be biased, less reproducible.

2. Analogous Estimation:

- **Description:** Uses historical data from similar projects to estimate effort and cost for a new project.
- **Merits:** Leverages past experience, can be accurate if similar projects are available.
- **Limitations:** Requires relevant historical data, may not account for differences in new projects.

3. Parametric Estimation:

- **Description:** Uses mathematical models with parameters derived from historical data to estimate effort and cost (e.g., COCOMO, SLIM).
- **Merits:** Objective, scalable, can handle complex relationships.
- **Limitations:** Requires accurate historical data, complex models can be difficult to understand and apply.

4. Bottom-Up Estimation:

- **Description:** Estimates effort and cost by breaking down the project into smaller components and estimating each component individually.
- **Merits:** Detailed, can be very accurate, allows for thorough understanding of the project.
- **Limitations:** Time-consuming, requires detailed knowledge of the project, can be complex.

5. Top-Down Estimation:

K. J. Somaiya College of Engineering, Mumbai-77

(Autonomous College Affiliated to University of Mumbai)

- **Description:** Estimates effort and cost by looking at the project as a whole and using high-level parameters and overall characteristics.
- **Merits:** Quick, requires less detailed information, useful in early stages.
- **Limitations:** Less detailed, may overlook specific project complexities, less accurate for detailed planning.

6. Wideband Delphi:

- **Description:** A structured group estimation technique where experts anonymously provide estimates, discuss them, and refine their estimates iteratively.
- **Merits:** Reduces bias, leverages group knowledge, improves accuracy through consensus.
- **Limitations:** Time-consuming, requires coordination, may still be subject to groupthink.

7. Function Point Analysis (FPA):

- **Description:** Measures software size by counting function points based on the functionality provided to the user, then uses this measure to estimate effort and cost.
- **Merits:** Independent of programming language, focuses on functionality, useful for early estimates.
- **Limitations:** Requires detailed analysis of requirements, subjective interpretation of functions.

8. Machine Learning and Artificial Intelligence:

- **Description:** Uses ML/AI models trained on historical project data to predict effort and cost.
- **Merits:** Can handle complex, non-linear relationships, improves over time with more data.
- **Limitations:** Requires large datasets, complex models can be difficult to interpret, dependent on data quality.